

Growing a classification tree using the apparent misclassification rate

E.Zintzaras, N.P.Brown and A.Kowald

Abstract

A method to determine the size of a classification tree is proposed. This method is based on the change of the apparent misclassification rate (AMR) of the tree at each growing stage. The method is simple and fast compared to the other classification tree methods, which are based on minimizing a cost complexity function. To test the method, it was used to classify species of fungi, and the results are in good agreement with those obtained by linear discriminant analysis. Also, 21 proteins with known structures and functions were classified using the proposed method. For this purpose the coefficient of variation for several properties of the secondary structures of these proteins has been used. Again, the results were in good agreement with the classification obtained previously using dynamic programming.

Introduction

The allocation of individuals into classes on the basis of measured variables is commonly performed using linear discriminant analysis (Fisher, 1936; Lachenbruch, 1975). This analysis assumes that the individuals come from normally distributed populations—an assumption that is often unrealistic. If the individuals can be discriminated, then they can also be clustered (Jardine and Sibson, 1971), in which case the identification of clusters is based on the distance between individuals. The problem with this method is the definition of the distance (Gordon, 1981; Krzanowski, 1988).

Breiman *et al.* (1984) described a non-parametric discrimination (classification) method. This method partitions the initial set of individuals recursively into subsets using one of the variables. For this the initial set is sorted according to each of the different (numerical) variables and then split into two subsets using different threshold levels for each variable. The variable and threshold that result in the best split (in terms of purity of classes) are finally chosen to split the set. When the procedure is applied recursively to the resulting subsets the variables and thresholds used at each branching point are independent of each other. This ensures that, independent of the number of variables, the algorithm always chooses the variable which is best suited for partitioning the set. Finally, the results are presented as a

tree structure, with the original set of individuals representing the root, and the different subsets branching off to form intermediate and, eventually, terminal nodes. A class is assigned to each terminal node using the majority rule, i.e. the class that is most numerous in the node is assigned.

The size of the tree is determined in the following way: a large tree is constructed and is then pruned upwards. The tree with the minimum cost complexity function is chosen. The cost complexity function is a function of the misclassification rate of the tree, calculated using cross-validation or resubstitution (Lachenbruch and Mickey, 1968) and the number of terminal nodes. This approach is computationally very intensive (Breiman *et al.*, 1984).

The tree can be used to classify and to identify groups of individuals at the same time, since individuals within the same branch are more similar than individuals from other branches.

In this paper, a simple and fast method is proposed in order to decide the size of the tree, based on the apparent misclassification rate (AMR) (Efron and Tibshirani, 1991) of the tree at each growing stage. The method is applied to a dataset of six species of fungi, the results being compared with those from conventional discriminant methods, such as linear discriminant analysis. The method is also used to classify and identify groups of proteins with known structures based on the coefficient of variation (CV) for several properties of the secondary structures of these proteins. The results are compared with the findings of Orengo *et al.* (1992) who used a dynamic programming method to study the same set of proteins.

Methods

If $x_1, x_2, \dots, x_k$ are the measurements made on an individual, the algorithm sorts the individuals according to each variable. It finds for each variable the threshold that gives the best split, i.e. the split that best separates the classes, compares these splits and chooses the best variable and split combination. The split separates the dataset into two subsets of individuals, each of which is then partitioned independently using the same procedure.

The partition algorithm is represented as a tree. The node t is split into two subnodes t_l and t_r in such a way that the subnodes are purer than the previous node, the proportion of the individuals in t that go to t_l and t_r are p_l and p_r , respectively. A node is pure when it contains only individuals from one class.

If a split occurs in variable x_k and the value of the variable at the threshold is S , then t_l will contain all the individuals

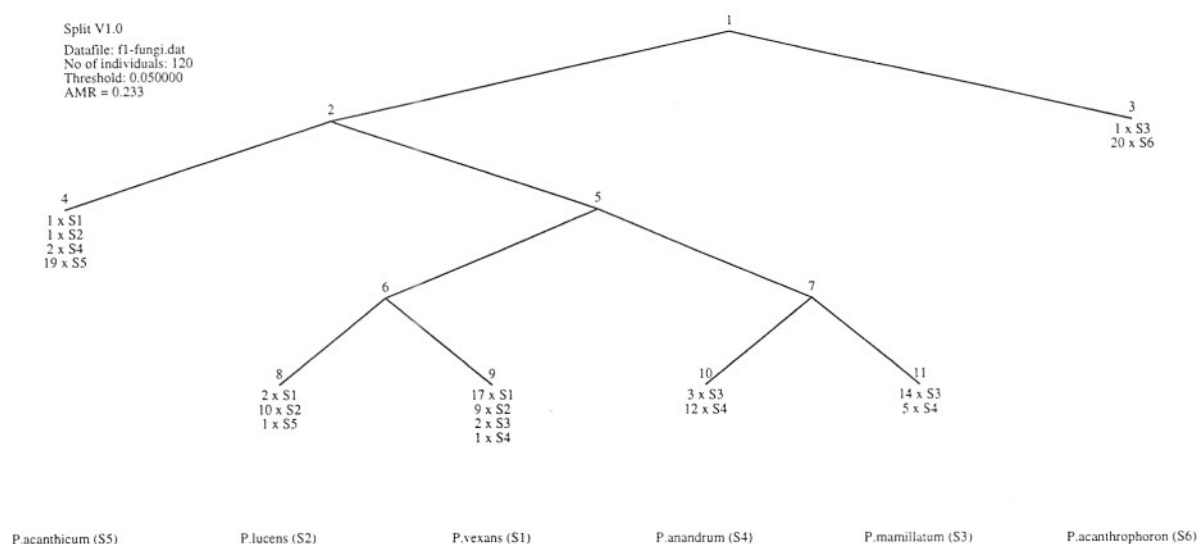


Fig. 1. The tree obtained for the six species of *Pythium*. The AMR was 23%. The threshold for splitting a node was based on a 5% improvement of the AMR. At each terminal node the number of individuals from each species is shown, e.g. in terminal node 3 there is one individual of *P.mamillatum* (1XS3) and 20 individuals of *P.acanthrophoron* (20XS6). At the bottom of the figure the class (species) which has been assigned to the terminal node is shown.

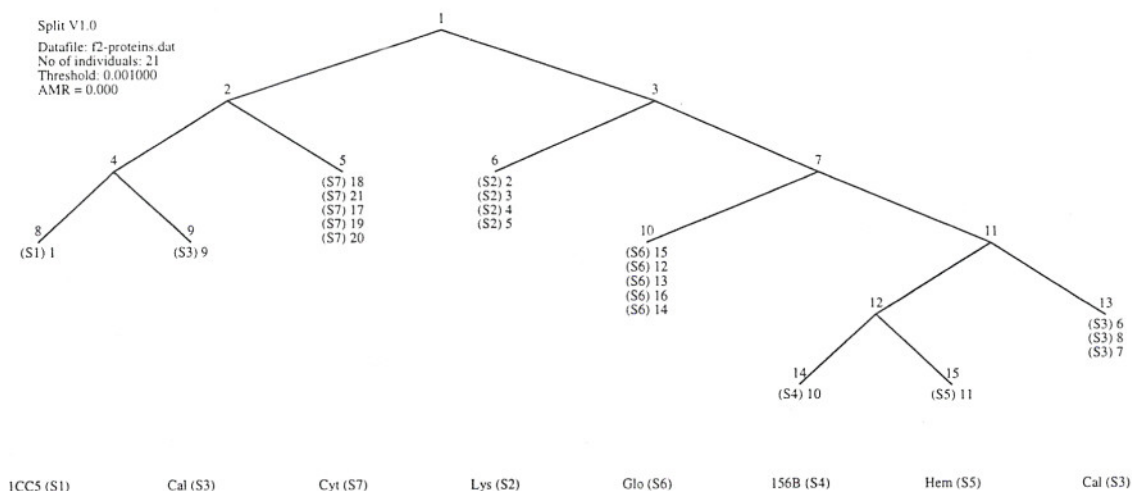


Fig. 2. The tree which resulted when the coefficients of variation were used to classify the proteins. The classification was perfect in the sense that the AMR was zero. However, instead of seven terminal nodes for seven classes there are eight. The 3ICB protein has been separated from the remaining calcium-binding proteins. Each protein in a terminal node is identified by an integer label listed in Table II.

which have values for x_k less than S , with the remainder in t_r . Breiman *et al.* (1984) defined a measure of the impurity of node t as a non-negative function of a vector of proportions to the interval (0,1). If 'c' is the number of different classes for the dataset the goodness, D , of the split is defined by the increase in purity due to split S . One measure of the node impurity is the Gini index of diversity ($I(t) = 1 - \sum_{i=1}^c [\pi(i/t)]^2$), where $\pi(a/b)$ is the proportion of individuals of class 'a' in node 'b'. Then the increase in purity due to the split S is given by:

$$DI(S) = I(t) - [p_l \cdot I(t_l) + p_r \cdot I(t_r)] \quad (1)$$

It can be seen that for pure nodes the summation terms become '1' and $DI(S)$ reaches a maximum.

After each split, classes are assigned to the terminal nodes using the majority rule, and the AMR of the tree is calculated. If the next split improves the AMR by a certain percentage, e.g. 5% (the program allows any value), that split is accepted, otherwise the node becomes a terminal node.

Implementation

The algorithm was programmed in C and currently runs on a Sun SPARCstation SLC (~19 MIPS) under SunOS 4.1.1 Unix. However, it is suitable for all systems that allow command line arguments and where a C compiler is available.

The name of a text datafile is supplied as the only command line argument. This datafile contains the number of variables,

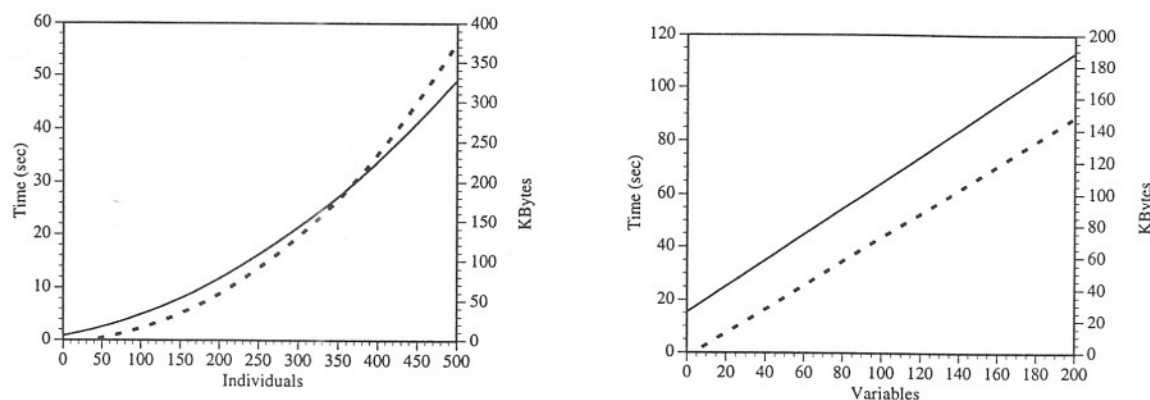


Fig. 3. The memory (—) and time (---) requirements for a varying number of individuals (left) and variables (right). In the simulation to the left the number of variables was kept constant (5) and in the simulation to the right the number of individuals was kept constant (100). As predicted (equations 2 and 3) both memory and time demands increase linearly with the number of variables and to the powers of two and three, respectively, with the number of individuals. The curves shown have been fitted to 100 experimental data points.

the number of different classes, the number of individuals, the names of the classes and, for each individual, a line with its variable values. The program is driven interactively and includes a facility for X-Window System (v. 11, tested on releases 4 and 5) graphical output identical to the examples in Figures 1 and 2. Although it is convenient to examine the result of a computation immediately on the terminal, an X environment is not essential. User interaction is text based and uses any VT100-compatible terminal (e.g. xterm). Instead of generating screen output, the user can produce a PostScript file containing the same graphical information.

As can be seen from Figures 1 and 2, two different types of graphical output can be specified: the condensed output (Figure 1), which summarizes for each terminal node the number of individuals that belong to different known classes; and the long output format (Figure 2), which lists all individuals of each terminal node.

In addition to the tree representation, which gives a visual impression of the structure of the data, textual output can be generated which lists for each node (i) the number of individuals, (ii) the variable chosen for the split, (iii) the individuals, (iv) a summary of the number of individuals belonging to different classes, and (v), for terminal nodes, a measure of their purity. The results for example 2 (see below) in this format have been placed in the Appendix.

While the datasets we are using here are relatively small, the algorithm is able to handle much larger datasets without problem. With I being the number of individuals, V the number of variables and S the number of species, the memory and time requirements are approximately given by:

$$\text{memory} \approx I^2 + I + I \cdot V \quad (2)$$

$$\text{time} \approx I^3 \cdot V + I^2 \cdot V \cdot S + I \cdot V + V S \quad (3)$$

The exact requirements depend on the number of nodes of

the tree, which in turn depends on the specific data used. These equations cover a worst-case scenario assuming that the tree contains the largest number of nodes possible. Note that for an increasing number of individuals, memory demand is quadratic with I and time required is cubic, while an increase in the number of variables (V) influences these demands only linearly. Figure 3 shows the results of simulations obtained using a SPARC SLC (~19 MIPS).

Example 1

Six species of fungi (genus *Pythium*) were classified using the proposed method. The results obtained by the method were compared with the results obtained by applying linear discriminant analysis.

Each species of fungi was represented by 20 individuals. For each individual, seven variables were measured: the means of the cross diameters of the oogonium, the oospore and ooplast, the thickness of the wall, the aplerotic index of the oogonium, the ooplast index and the wall index of the oospore (Sumati and Dick, 1988; Zintzaras, 1990; Shahzad *et al.*, 1992). The individuals were classified based on the above variables using the AMR tree method. The obtained tree is shown in Figure 1. The tree discriminated the six species with an AMR of 23%.

The individuals segregated into six terminal nodes. The terminal nodes assigned to species *P. acanthicum* and *P. acanthrophoron* are very pure, while the other nodes contain some degree of species mixing. For instance nine individuals of species *P. lucens* are mixed with individuals of species *P. vexans*. However, linear discriminant analysis gave similar results to the tree (Table I). Most individuals were classified into their species, and some individuals of *P. lucens* are mixed together with individuals of *P. vexans*.

Thus, the AMR tree non-parametric method can be used for classifying individuals into their species based on a set of measurements on these individuals.

Table I. The results of the linear discriminant analysis on the dataset from the six species of *Pythium*

From species	No. of individuals classified into species					
	s1	s2	s3	s4	s5	s6
s1	13	7	1	0	0	0
s2	5	12	0	0	0	0
s3	1	0	15	0	0	0
s4	0	0	4	18	0	0
s5	1	1	0	2	20	0
s6	0	0	0	0	0	20
No. of correct classified	13	12	15	18	20	20

s1 = *P.vexans*, s2 = *P.lucens*, s3 = *P.mamillatum*, s4 = *P.anandrum*, s5 = *P.acanthicum*, s6 = *P.acanthrophoron*. The AMR was 18%.

Table II. The groups of 21 proteins as clustered by Orengo *et al.* (1992)

Group	Protein	Label
Lysozymes	1CC5	1
	1ALC	2
Calcium binding	2LZ2	3
	1LZT	4
	1LZ1	5
	3CLN	6
	2CLN	7
	5TNC	8
	3ICB	9
156B	156B	10
Hemerythrins	2MHR	11
	2LH2	12
Globins	1MBS	13
	2LHB	14
Cytochromes c	1ECA	15
	1MBN	16
	351C	17
	1CYC	18
	155C	19
	3C2C	20
	1CCR	21

Example 2

Orengo *et al.* (1992) describe a method to compare protein structures. Structural data are used to align the protein sequences and, to speed up the computationally demanding dynamic programming algorithm, initial alignments based on distance and angles between the secondary structures (α -helices and β -sheets) in each protein are used in a fast pre-filter stage before the main alignment. The resulting alignment scores are used to cluster the proteins according to structural similarity.

A subset of their results, namely groupings for all- or predominantly α -helix containing proteins, are shown in Table II. A variety of geometric and physico-chemical measurements on secondary structure pairs based on Orengo *et al.* (1992) and Brown (1992) were used in the AMR classification of these proteins:

v_1 = protein size as number of possible pairs of secondary structures

v_2 = distance between the centres of two secondary structures

v_3 = distance between ends of secondary structures

v_4 = intercentroid tilt angles between two structures

v_5 = intercentroid angles between two structures

v_6 = hydrophobic moment vector angle

v_7 = centroid moment vector angle

v_8 = connecting loop length as number of amino acid residues between secondary structures

From these measurements, new variables were derived that were applied in classifying proteins by the AMR tree method. For the first variable (v_1) a simple size measure was defined as the number of possible interactions between secondary structures of the protein [$v_1 = n(n-1)/2$]. Additionally, seven other variables were used and derived from the original data in the following way. For each protein the mean and the standard deviations of $v_2, \dots, v_8$ were calculated. Then the standard deviations were divided by the corresponding means, the result of this division being termed the CV. The CV shows the size of variation relative to the size of the measurement, and is independent of the units of measurement.

For the proteins shown in Table II the described variables have been derived and the obtained tree is shown in Figure 2. With a threshold level of 0.1% a perfect tree with no misclassified proteins resulted (i.e. the AMR was zero). A listing of the nodes of the tree is given in the Appendix. As described under Implementation, this representation allows a more detailed analysis of the results.

The results obtained with the classification tree are in good agreement with the results of Orengo *et al.* (1992). The main groups of lysozymes, globins, cytochromes and calcium-binding proteins are reproduced at terminal nodes, reflecting each group's distinctive internal structural organization in terms of the properties $v_1, \dots, v_8$. However, while three calcium-binding proteins were placed together in one terminal node, 3ICB was separated from the other calcium-binding proteins. The reason for this is that 3CLN, 2CLN and 5TNC contain two major domains, whereas 3ICB only possesses one. This accounts for a considerable difference in size which causes the separation of the calcium-binding proteins into different groups.

The AMR tree approach seems effective in regenerating the same structural groupings as the method of Orengo *et al.* (1992), so that proteins may be classified using the above derived variables in combination with the tree method, which could also accommodate the other properties used in their study. The AMR tree method could provide a rapid pre-filter to a more detailed topological analysis using the method of Orengo *et al.* (1992).

Discussion

A new method for determining the size of the tree of the non-parametric classification method described by Breiman *et al.* (1984) has been proposed. This classification method has the form of a tree branching off into intermediate and terminal

nodes. Using specific variables for each individual, each split generates subnodes which are purer than the parent node. The new algorithm for controlling the size of the tree is based on the improvement of the AMR after each split. This approach makes the construction of a tree simple, and therefore faster than minimizing a cost complexity function (Lachenbruch and Mickey, 1968).

The proposed algorithm was implemented as a C program and applied to two examples. The first example included 120 individuals of fungi belonging to six different species and the tree method successfully classified the individuals into their species. To assess the quality of the classification a linear discriminant analysis was performed on the data and both results are in good agreement.

As a second example a set of 21 proteins has been classified. For this purpose variables had to be derived which could be used with the AMR tree method. It was proposed that the CV of several properties of the secondary structures of these proteins might be used for classifying and identifying groups of proteins with the AMR tree method. The results coincide very well with the classification of Orengo *et al.* (1992) using dynamic programming.

The method is fast and simple, and can therefore be used on less powerful machines. Furthermore it gives a graphical representation of the structure of the data and gives results that are in good agreement with studies performed using more complicated and time-consuming procedures.

Acknowledgements

We thank Dr T. Kirkwood and Dr W.R. Taylor for comments on the paper and Professor J. Gower and Dr P. Taylor for useful discussions on the subject.

References

- Breiman, L., Friedman, J.H., Olson, R.A. and Stone, C.J. (1984) *Classification and Regression Trees*. Wadsworth, Belmont, CA.
- Brown, N.P. (1992) Patterns in protein secondary structure packing: a database for prediction. Ph.D. thesis, National Institute for Medical Research, Mill Hill, London.
- Efron, B. and Tibshirani, R. (1991) Statistical data analysis in the computer age. *Science*, **253**, 390–395.
- Fisher, R.A. (1936) The use of multiple measurements in taxonomic problems. *Ann. Eug.*, **7**, 179–184.
- Gordon, A.D. (1981) *Classification*. Chapman & Hall, London.
- Jardine, N. and Sibson, R. (1971) *Mathematical Taxonomy*. John Wiley, London.
- Krzyszowski, W.J. (1988) *Principles of Multivariate Analysis*. Clarendon Press, Oxford.
- Lachenbruch, P.A. (1975) *Discriminative Analysis*. Hafner, NY.
- Lachenbruch, P.A. and Mickey, M.R. (1968) Estimation of error rates in discriminant analysis. *Technometrics*, **10**, 1–11.
- Orengo, C.A., Brown, N.P. and Taylor, W.R. (1992) Fast structure alignment for protein databank searching. *Proteins: Struct. Funct. Genet.*, **14**, 139–167.
- Shahzad, S., Coe, R. and Dick, M.W. (1992) Biometry of oospores and oogonia of *Pythium* (oomycetes)—the independent taxonomic value of calculated ratios. *Bot. J. Linn. Soc.*, **108**, 143–165.
- Sumati, B. and Dick, M.W. (1988) Biometry of oogonia and oospores of *Saprolegniaceae*. *Bot. J. Linn. Soc.*, **100**, 219–235.
- Zintzaras, E. (1990) Biometry of fungi. M.Sc. thesis, University of Reading.

Received June 18, 1993; accepted December 13, 1993